Comparación experimental del rendimiento de los algoritmos de ordenamiento por inserción y Shellsort

Fabrizio Rigio Apaza Vizcarra
Universidad Nacional Jorge Basadre Grohmann
Escuela Profesional de Ingeniería en Informática y Sistemas

Resumen

En este trabajo se realiza una comparación experimental del rendimiento de los algoritmos de ordenamiento por inserción y Shellsort. Para ello, se evaluó el comportamiento de ambos algoritmos utilizando arreglos de diferentes tamaños y tres patrones de entrada: aleatorio, ascendente y descendente. Los experimentos se realizaron con tamaños de 100, 1000 y 10000 elementos, efectuando 30 repeticiones para cada combinación de algoritmo, tamaño y patrón. Como métricas de evaluación se consideraron el tiempo de ejecución, el número de comparaciones y el número de movimientos realizados durante el ordenamiento. Los resultados obtenidos permiten analizar cómo el tamaño y la distribución inicial de los datos influyen en el rendimiento de cada algoritmo. La comparación experimental muestra diferencias importantes entre ambos métodos, especialmente cuando aumenta el tamaño de los datos y estos presentan un orden desfavorable para el algoritmo de inserción.

1. Introducción

El ordenamiento de datos es una operación fundamental en el campo de las ciencias de la computación, debido a que permite organizar información de manera eficiente y facilita posteriormente procesos como la búsqueda, el análisis y el procesamiento de datos. Existen diversos algoritmos de ordenamiento, cada uno con características y comportamientos diferentes frente al tamaño y la distribución inicial de los elementos. Por esta razón, el análisis experimental de estos algoritmos permite observar cómo sus características teóricas se reflejan en situaciones prácticas.

Entre los algoritmos de ordenamiento se encuentra el método de inserción, cuya estrategia consiste en recorrer los elementos de un arreglo y colocar cada elemento en la posición correspondiente dentro de la parte previamente ordenada. Su implementación es sencilla y puede presentar un comportamiento favorable cuando los datos se encuentran ordenados o parcialmente ordenados. Sin embargo, cuando el tamaño de entrada aumenta y los elementos presentan una distribución desfavorable, el número de comparaciones y movimientos puede crecer considerablemente.

Shellsort representa una mejora del método de inserción, ya que utiliza intervalos o saltos que permiten comparar elementos que se encuentran alejados entre ellos. Estos intervalos descienden sucesivamente hasta llegar a uno, permitiendo que el arreglo se encuentre con mas orden antes de realizar la última etapa de inserción. Esta estrategia permite reducir parte del trabajo que tendría que realizar directamente el método de inserción sobre un arreglo altamente desordenado.

Diversos estudios sobre algoritmos de ordenamiento han mostrado que el rendimiento de un algoritmo depende tanto de su estrategia como de las características de los datos de entrada. En particular, los

métodos basados en inserción pueden presentar diferencias importantes entre entradas ordenadas, aleatorias y en orden inverso. Shellsort, por su parte, busca disminuir este efecto mediante el uso de intervalos que permiten realizar movimientos de mayor alcance.

En este contexto, el presente trabajo tiene como objetivo comparar experimentalmente el rendimiento de los algoritmos de ordenamiento por inserción y Shellsort utilizando diferentes tamaños y patrones de entrada. Para ello, se emplearon arreglos de 100, 1000 y 10000 elementos, considerando datos aleatorios, ascendentes y descendentes. Cada condición experimental fue ejecutada 30 veces y se registraron como métricas el tiempo de ejecución, el número de comparaciones y el número de movimientos.

Como hipótesis descriptiva, se asume que Shellsort presentará un mejor rendimiento general que el método de inserción conforme aumente el tamaño de los datos, especialmente en entradas aleatorias y descendentes. De esta forma, se espera que el método de inserción presente un comportamiento favorable cuando los datos se encuentren desde el inicio ordenados, debido a que requiere un menor número de movimientos para mantener el orden.

2. Métodos

2.1. Diseño experimental

Realize un experimento comparativo entre los algoritmos de ordenamiento por inserción y Shellsort. El propósito fue evaluar su comportamiento frente a diferentes tamaños y patrones de entrada, utilizando métricas relacionadas con el costo computacional y su tiempo de ejecución.

El experimento consideró tres tamaños de entrada: 100, 1000 y 10000 elementos. Para cada tamaño se utilizaron tres patrones de datos: aleatorio, ascendente y descendente. De esta manera, se evaluaron nueve condiciones de entrada para cada algoritmo. Cada condición fue ejecutada 30 veces, obteniéndose un total de 540 ejecuciones correspondientes a los dos metodos utilizados.

2.2. Algoritmos evaluados

El primer metodo utilizado fue el método de ordenamiento por inserción. Este algoritmo recorre el arreglo desde el segundo elemento y compara cada elemento con los elementos anteriores que ya se encuentran ordenados. Cuando es necesario, los elementos mayores se desplazan una posición para insertar el elemento en su posición correspondiente.

El segundo algoritmo evaluado fue Shellsort. Este método puede considerarse una mejora del ordenamiento por inserción, debido a que inicialmente compara elementos separados mediante un intervalo o *gap*. El intervalo comienza con un valor mayor y se reduce sucesivamente hasta que llegue a uno. Como ultima proceso se aplica un proceso equivalente al ordenamiento por inserción sobre el arreglo.

Los dos algoritmos fueron desarrollados en C++ empleando la misma estructura de arreglos de numeros enteros. Para asegurar una evaluaciopn equitativa, ambos algoritmos fueron alimentados con identicos datos de entrada en cada ejecucion.

2.3. Generación de datos

La información empleada en el experimento consitió en arreglos de numeros enteros generados nediante tres patrones distintos. El patrón aleatorio genero valores enteros mediante la funcion de genera-

cion pseudoaleatoria. El patrón ascendente organizo elementos de menor a mayor, mientras que el patrón descendente lo organizo en orden contrario.

Para cada repetición se generó un arreglo original y se realizó una copia del mismo antes de ejecutar cada algoritmo. De esta manera, tanto inserción como Shellsort procesaron exactamente la misma entrada, evitando que las diferencias en los datos afectaran la comparación experimental.

2.4. Variables y métricas

Las principales variables independientes del experimento fueron el algoritmo utilizado, el tamaño del arreglo y el patrón de entrada. Las variables dependientes fueron el tiempo de ejecución, el número de comparaciones y el número de movimientos realizados durante el ordenamiento.

El tiempo de ejecución se midió en milisegundos utilizando las herramientas de medición de tiempo disponibles en C++. El número de comparaciones correspondió a las comparaciones realizadas entre elementos durante el proceso de ordenamiento. Los movimientos contabilizaron los desplazamientos y colocaciones de elementos realizados por cada algoritmo.

Al finalizar cada ejecución se verificó que el arreglo estuviera correctamente ordenado. Seguidamente, los resultados se almacenaron en un archivo CSV para facilitar su procesamiento y análisis estadístico.

2.5. Repeticiones y procesamiento de resultados

Cada combinación de algoritmo, tamaño y patrón fue ejecutada 30 veces. Los resultados individuales fueron almacenados para despues calcular los valores promedio de tiempo de ejecución, comparaciones y movimientos.

En total, se obtuvieron 540 registros experimentales, correspondientes a 2 algoritmos, 3 tamaños, 3 patrones y 30 repeticiones. Los resultados promedio fueron organizados posteriormente para facilitar la elaboración de tablas y figuras utilizadas en el análisis.

Para garantizar que el experimento funcionara, se utilizaron semillas controladas para la generación de los datos aleatorios. Se empleó una semilla diferente para cada repetición, permitiendo regenerar las mismas entradas bajo las mismas condiciones. Además, cada registro de resultados incluye el `commit_hash` correspondiente a la versión del código utilizada durante la ejecución, permitiendo identificar de manera precisa la implementación evaluada. Los resultados los registre en archivos CSV para su posterior procesamiento y análisis.

3. Resultados

Los resultados obtenidos corresponden a las 540 ejecuciones realizadas durante el experimento, considerando los algoritmos de inserción y Shellsort, tres tamaños de entrada y tres patrones de datos. Para cada condición experimental se calcularon los valores promedio del tiempo de ejecución, número de comparaciones y número de movimientos. Estos resultados permiten observar las diferencias de comportamiento entre ambos algoritmos a medida que aumenta el tamaño de los datos y cambia su distribución inicial.

3.1. Tiempo de ejecución

La Tabla 1 presenta los tiempos promedio de ejecución obtenidos para los algoritmos de inserción y Shellsort bajo los tres patrones de entrada: aleatorio, ascendente y descendente. Los tiempos se expresan en milisegundos y corresponden al promedio de las 30 repeticiones realizadas para cada condición experimental.

Cuadro 1: Tiempo promedio de ejecución en milisegundos.

Algoritmo	Patrón	$n = 100$	$n = 1000$	$n = 10000$
Inserción	Aleatorio	0.0333	0.6009	69.1621
Inserción	Ascendente	0.0000	0.0000	0.0000
Inserción	Descendente	0.0000	1.2007	136.4345
Shellsort	Aleatorio	0.0000	0.0667	2.3333
Shellsort	Ascendente	0.0000	0.0000	0.6000
Shellsort	Descendente	0.0000	0.0333	0.5999

Las Figuras 1, 2 y 3 muestran la evolución del tiempo promedio conforme incrementa el tamaño de la entrada. Se puede observar como Shellsort presenta un menor tiempo de ejecución que Inserción, especialmente para $n = 10000$.

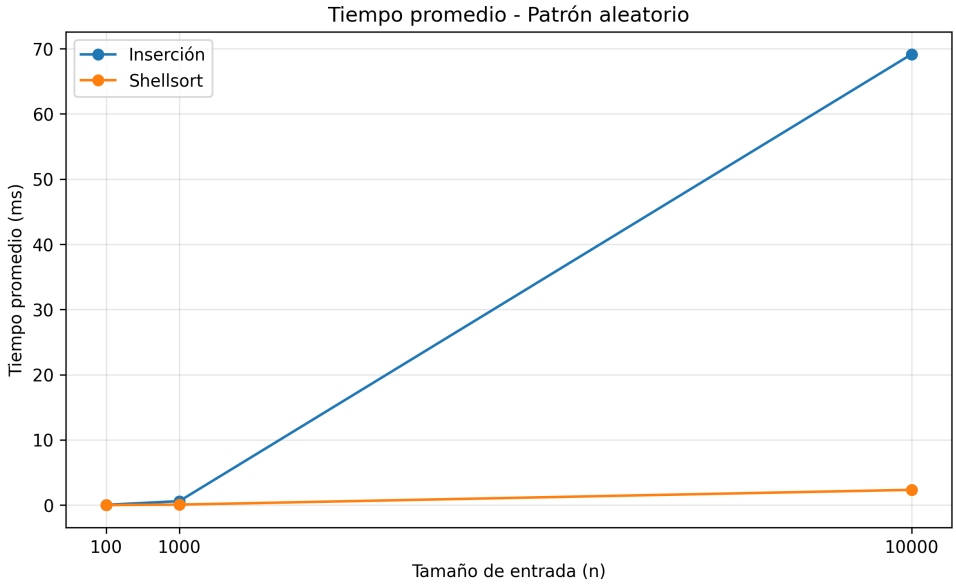


Figura 1: Tiempo promedio para el patrón aleatorio.

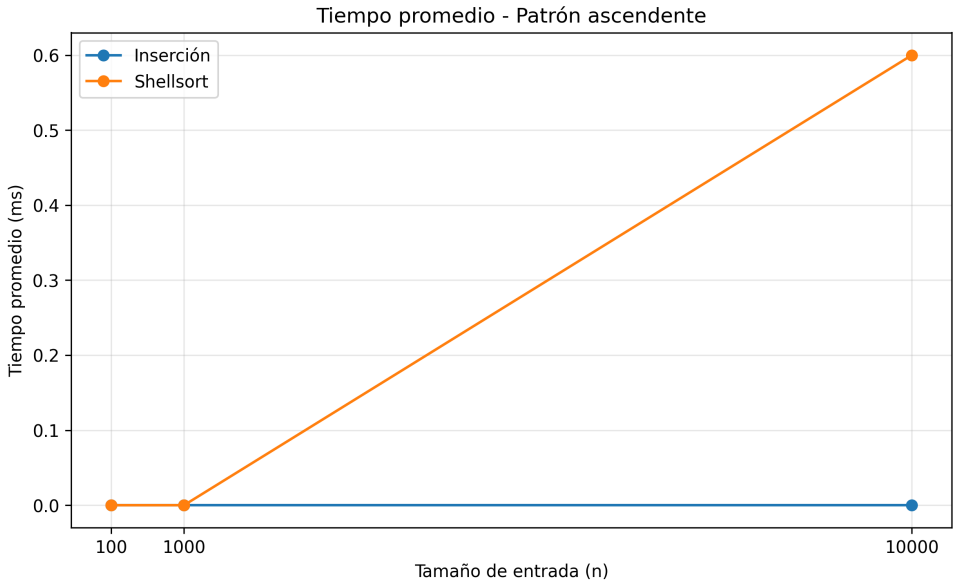


Figura 2: Tiempo promedio para el patrón ascendente.

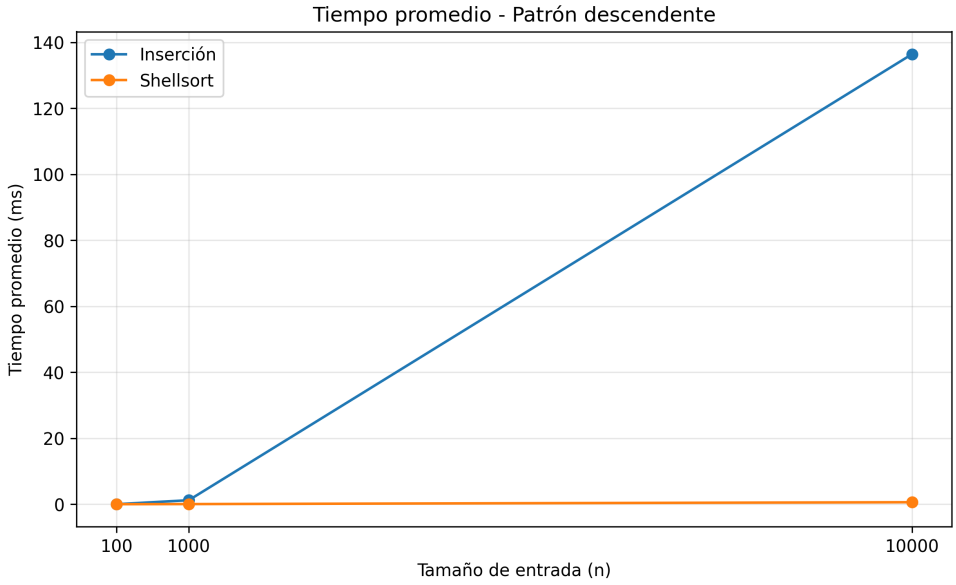


Figura 3: Tiempo promedio para el patrón descendente.

3.2. Número de comparaciones

La Tabla 2 presenta el número promedio de comparaciones realizadas por ambos algoritmos durante el proceso de ordenamiento. Esta métrica permite analizar la cantidad de operaciones de comparación necesarias para procesar los diferentes tamaños y patrones de entrada.

Cuadro 2: Número promedio de comparaciones realizadas.

Algoritmo	Patrón	$n = 100$	$n = 1000$	$n = 10000$
Inserción	Aleatorio	2550.97	250268.03	24969816.10
Inserción	Ascendente	99.00	999.00	9999.00
Inserción	Descendente	4950.00	499500.00	49995000.00
Shellsort	Aleatorio	860.13	15236.67	264261.27
Shellsort	Ascendente	503.00	8006.00	120005.00
Shellsort	Descendente	668.00	11716.00	172578.00

Las Figuras correspondientes permiten observar que Shellsort requiere considerablemente menos comparaciones que Inserción a medida que aumenta el tamaño de la entrada.

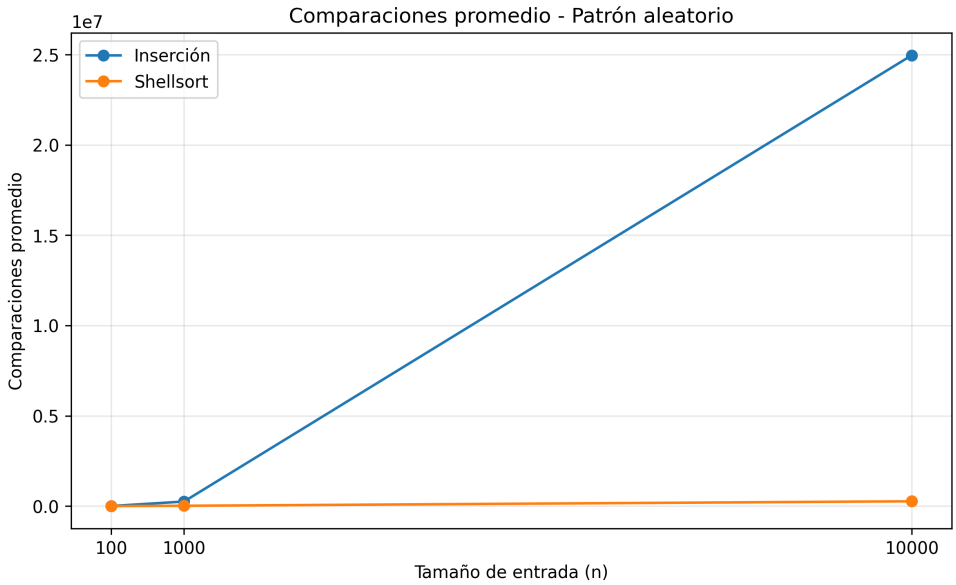


Figura 4: Comparaciones promedio para el patron aleatorio.

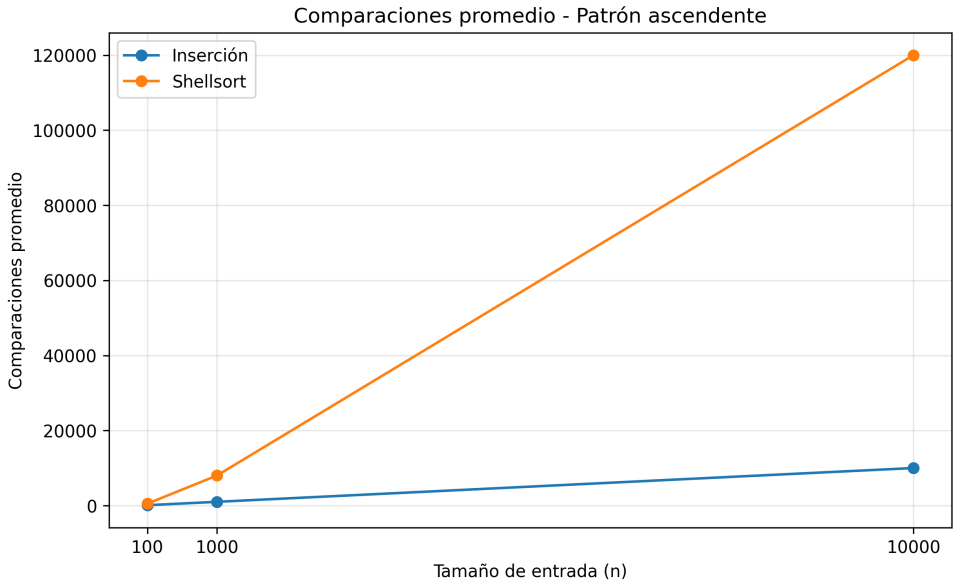


Figura 5: Comparaciones promedio para el patron ascendente.

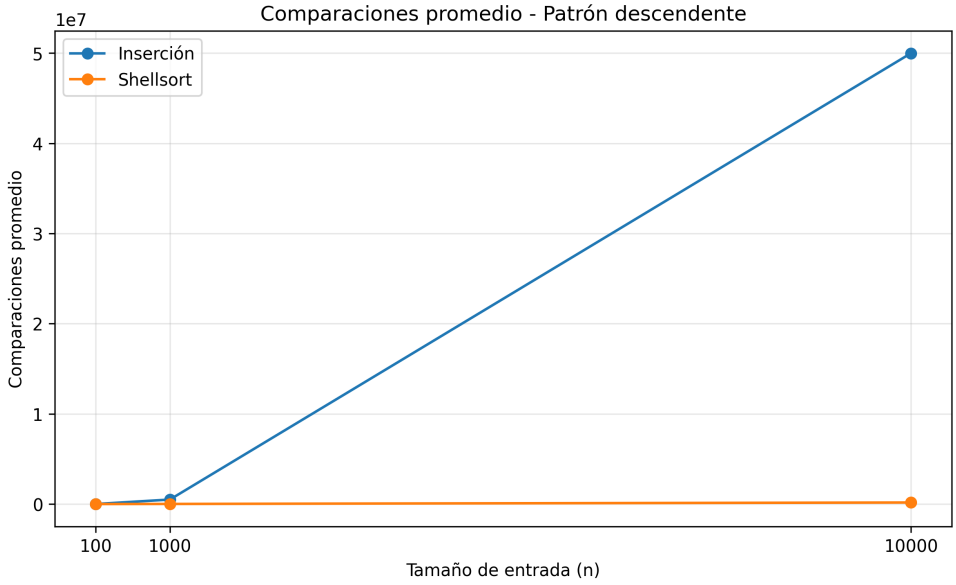


Figura 6: Comparaciones promedio para el patron descendente.

3.3. Número de movimientos

La Tabla 3 presenta el número promedio de movimientos realizados por los algoritmos de inserción y Shellsort. Los movimientos corresponden a los desplazamientos y colocaciones efectuados durante el proceso de ordenamiento.

Cuadro 3: Número promedio de movimientos realizados.

Algoritmo	Patrón	$n = 100$	$n = 1000$	$n = 10000$
Inserción	Aleatorio	2554.37	250273.90	24969824.43
Inserción	Ascendente	99.00	999.00	9999.00
Inserción	Descendente	5049.00	500499.00	50004999.00
Shellsort	Aleatorio	911.03	15751.20	269333.67
Shellsort	Ascendente	503.00	8006.00	120005.00
Shellsort	Descendente	763.00	12706.00	182565.00

Las figuras de movimientos permiten comparar el comportamiento de ambos algoritmos y observar cómo aumenta la cantidad de operaciones conforme crece el tamaño de la entrada.

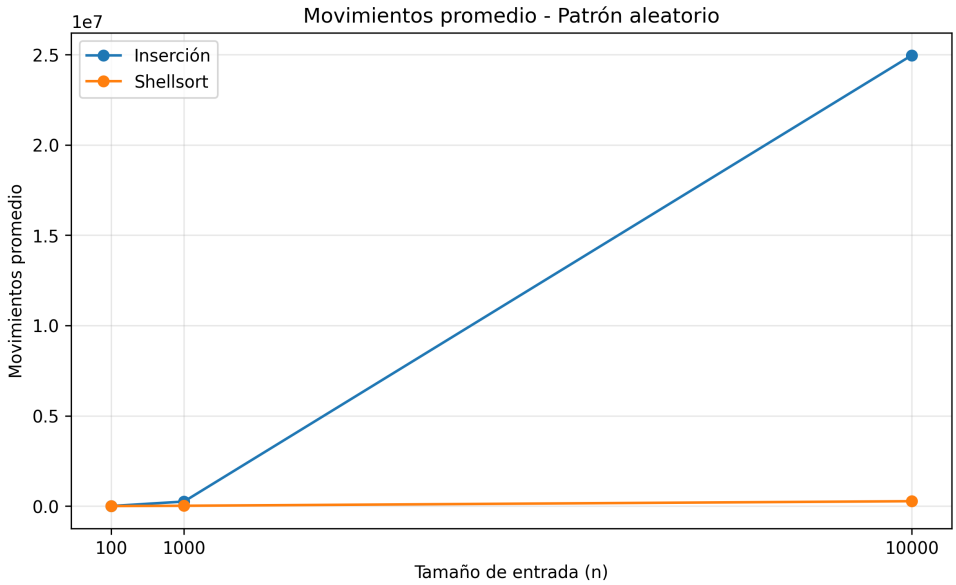


Figura 7: Movimientos promedio para el patrón aleatorio.

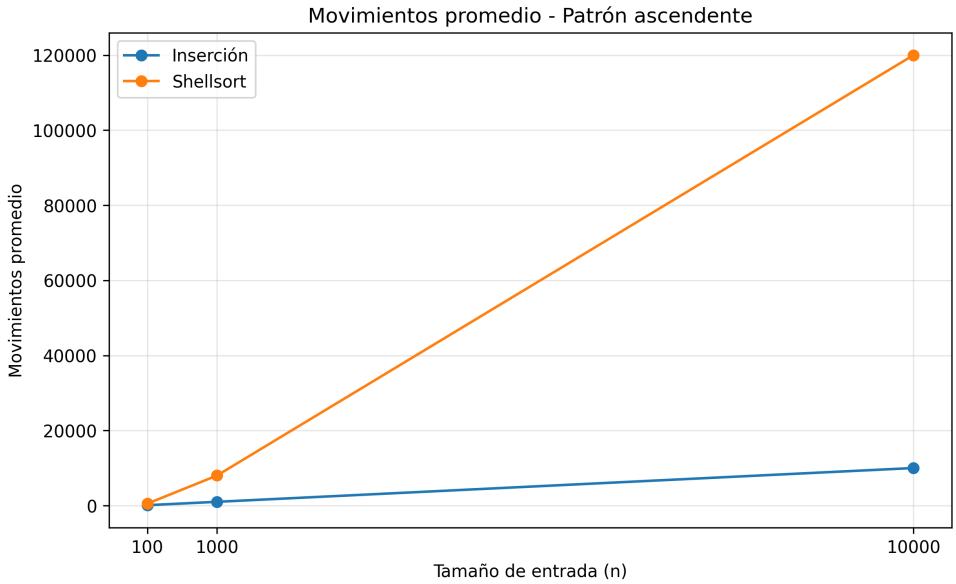


Figura 8: Movimientos promedio para el patrón ascendente.

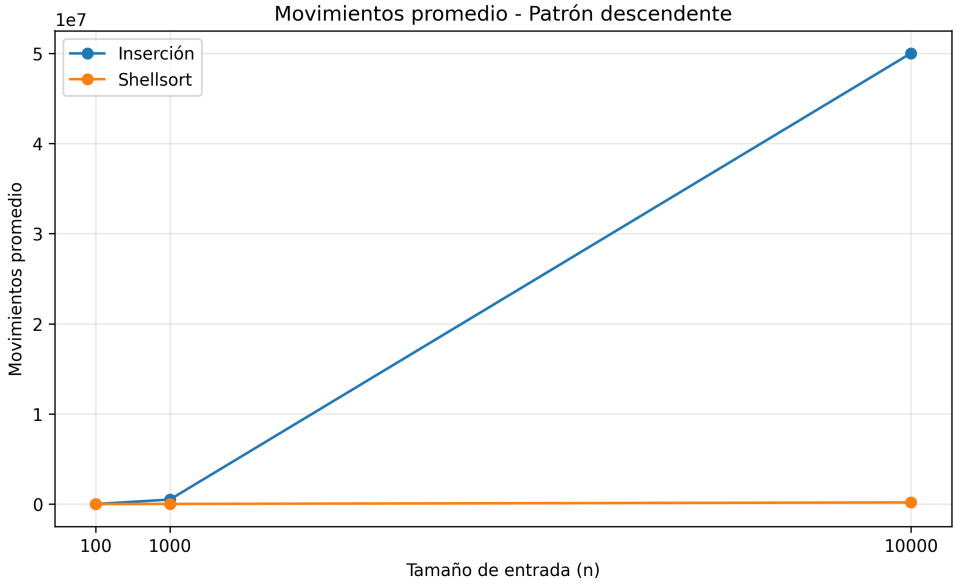


Figura 9: Movimientos promedio para el patrón descendente.

4. Discusión

Los resultados obtenidos muestran una diferencia significativa en el rendimiento entre el algoritmo de ordenamiento por inserción y Shellsort. En general, Shellsort presentó menores tiempos de ejecución, menor número de comparaciones y menor cantidad de movimientos, especialmente cuando el tamaño de la entrada aumentó. Esta diferencia se hizo más evidente para $n = 10000$, donde el algoritmo de inserción mostró un crecimiento considerable de su tiempo de ejecución, mientras que Shellsort mantuvo un comportamiento más eficiente.

El comportamiento observado coincide con las cualidades teóricas de ambos algoritmos. El método

de inserción puede alcanzar una complejidad de $O(n^2)$ en el peor caso, debido a que debe manipular una gran cantidad de comparaciones y desplazamientos cuando los elementos se encuentran en un orden desfavorable. Esto se observó principalmente en el patrón descendente, donde para $n = 10000$ se registraron aproximadamente 49.99 millones de comparaciones y 50.00 millones de movimientos. En contraste, Shellsort reduce la cantidad de operaciones mediante el uso de intervalos, " saltos" o "gaps", permitiendo comparar y desplazar elementos que se encuentran alejados entre sí.

El patron de entrada influyo de manera significativa en como funcionaron los algoritmos. En el caso del patron ascendente, insercion mostro un desempeño favorable, dado que los elementos estaban ya organizados y solo requeria hacer una comparacion por cada uno. No obstante, esta ventaja se desvanecio cuando los datos estaban dispuestos en orden decreciente o de forma aleatoria. Shellsort, en cambio demostro un comportamiento mas constante entre los distintos patrones, sobre todo para tamaños de entrada mayores.

En términos generales, los resultados indican que el algoritmo de inserción puede ser adecuado para conjuntos de datos pequeños o que ya se encuentran ordenados, debido a su simplicidad y bajo costo en estos casos. Sin embargo, cuando aumenta el tamaño de los datos o existe un orden desfavorable, Shellsort resulta más conveniente debido a que reduce considerablemente el número de operaciones necesarias. Por ello, los resultados experimentales respaldan la ventaja de Shellsort frente a inserción para entradas de mayor tamaño y diferentes condiciones de ordenamiento.

5. Conclusiones

Despues de observar los resultados obtenidos,estos nos permiten concluir que Shellsort presentó un mejor rendimiento general que el algoritmo de inserción en las condiciones evaluadas. Esta ventaja fue muy evidente al aumentar el tamaño de la entrada hasta $n = 10000$, donde Shellsort registró menores tiempos de ejecución, comparaciones y movimientos.

El método de inserción evidenció un desempeño positivo cuando los valores estaban organizados de forma ascendente, ya que necesitó menos acciones en esta situación. No obstante, su eficacia se vio notablemente afectada con datos al azar y, especialmente, con datos dispuestos en orden descendente, donde las comparaciones y traslados crecieron considerablemente.

Por otro lado, Shellsort presentó un comportamiento más estable frente a los diferentes patrones de entrada. El uso de intervalos permitió reducir la cantidad de operaciones necesarias para ordenar los elementos, logrando mejores resultados conforme aumentó el tamaño de los datos.

Finalmente, el experimento permitió comprobar de manera práctica las distinciones entre ambos algoritmos y mostrar cómo la complejidad y el orden inicial de los datos influyen en su eficiencia. Por lo tanto, aunque inserción puede ser una alternativa sencilla y adecuada para conjuntos pequeños o previamente ordenados, Shellsort resulta una opción más eficiente para entradas de mayor tamaño y con diferentes condiciones de ordenamiento.

6. Bibliografia

Sznajdleder, P. (2017). Programación orientada a objetos y estructura de datos a fondo. Alpha Editorial.

Ruiz, R. E. S., Rodríguez, J. E. R. (2005). Análisis de complejidad algorítmica. Revista Vínculos, 1(2), 3-12.

Islam, M. S., Halder, P. (2025). Unifying Algorithmic Performance: Insertion Sort Across Diverse Programming Paradigms.

AL-AMIN, I. M., Okeyinka, A. E., IBRAHIM, A. (2024). Comparative complexity study of bubble sort and insertion sort using java programming language: A review. Journal of Science Innovation and Technology Research.

Bustos, B. Estudio y optimización del algoritmo de ordenamiento Shellsort.

Salazar, J. Algoritmos de ordenación.

Sitorus, Z., Prayogi, D., Rizko, M. A., Suteja, A. G., Harahap, M. R. (2024). Implementation of the Insertion Sort Algorithm to Sort Positive Integers in Ascending Order Using Flowgorithm. Journal of Information Technology, Computer Science and Electrical Engineering, 1(3), 323-328.

Kalaivani, A., Swetha, K. (2021). An enhanced bidirectional insertion sort over classical insertion sort. International journal of image and graphics, 21(02), 2150024.